RoboCup Internationals - Rescue Maze 2026

Team Description Paper

League Name:	RCJ RESCUE MAZE
Age Group:	Secondary
Team Name:	The Worriers
Team Website:	Worriersrcj
Participants:	• Abhijit Sharma • Aaryaman Talwar • Jasjyot Singh Sethi • Kashvi Khajanchi
Document:	Team Description Paper
Mentor Name:	Mr. Vivek Gautum
Institution:	The Makers Robotics
Region:	INDIA
Contact Email:	vivekgautam51@gmail.com

INTENT

Our team, THE WORRIERS, is dedicated to advancing the frontiers of robotics and collaboration by participating in **RoboCup Junior Internationals 2026** for the category Rescue Maze. We are a group of three students from diverse backgrounds, united by our passion for **problem-solving through the integration of AI and programming.** While participating in the rescue Maze mission our intent was to innovate and construct a robot capable of analysing complex situations and performing rescue operations with **precision, reliability and ensuring the safety of the target keeping the sustainability aspect at the heart of our innovation.** For sustainability we strictly use only reusable plastic and have equipped our lab with e-waste collection box. We keep raising **awareness and conduct e-waste and plastic collection drives to reduce our carbon footprints.** We have a tie up with NAMO e-waste Management Ltd. for responsible disposal of e-waste. We believe in the positive outcome of the power of teamwork, collaboration, continuous learning, and inspiring one another to achieve our objectives. We are committed to excellence and a spirit of competition and hope to exhibit our efforts while making a meaningful impact in not only the world of robotics but its immense uses beyond. Together, we are excited to tackle the challenges ahead and contribute to the future of Rescue Robotics!

Benches made for community by
Plastic collection drives
June 2024- August 2025.
400 Kgs Plastic collected

E-Waste collection Bin placed in our Robotics lab

INDEX

Section	Title	Page
	INTENT	**2**
	ABSTRACT	**4**
1	**INTRODUCTION**	**4**
2	**PROJECT PLANNING**	**5**
2a	Project Timeline	6
3	**SYSTEM ARCHITECTURE**	**7**
4	**HARDWARE**	**8**
4a	Chassis and Drive	8
4b	Electronics and Wiring	9
4c	Power System	9
4d	Sensor Suite	9
5	**SOFTWARE**	**11**
5a	Sensor Acquisition	11
5b	Motion Primitives & Heading Control	11
5c	Wall Following	11
5d	Blind Odometry	11
5e	Ramps	11
5f	Mapping & Exploration	12
5g	Routing — Turn-Aware Dijkstra	12
5h	Hazard & Tile-Colour Handling	13
5i	Lack of Progress	13
6	**INNOVATIONS**	**14**
7	**PERFORMANCE EVALUATION AND CONCLUSION**	**14**
8	**ACKNOWLEDGEMENTS**	**15**
9	**APPENDIX**	**16**

ABSTRACT

Our team aims to develop a robot that operates with exceptional targeted precision, rescuing victims and overcoming all obstacles in accordance with the Robocup Junior Rescue Maze Rules .. What makes it unique is a strict three-tier **body / mind / dev** architecture: a Cytron Motion 2350 Pro (RP2350) handles all real-time sensing and closed-loop motion primitives, a Raspberry Pi 5 runs the mapping, frontier exploration and turn-aware Dijkstra routing, and a laptop is used only for development. The two boards speak a simple ACK-gated ASCII protocol over UART, so exactly one command is ever in flight and every motion is verified by sensors rather than assumed.

Six VL53L0X time-of-flight sensors (four for mapping, two dedicated to wall-following), an AS7341 spectral sensor for floor colour, and a BNO055 IMU sit behind a PCA9548A I²C multiplexer. A ~70 Hz control loop (rebuilt from an original 5 Hz blocking design) drives absolute-cardinal turns, calibrated single-tile moves with residual-error carry, PD wall-following, a blind-odometry fallback for long corridors, and IMU-based ramp traversal across multiple floors. The robot persists its map at every silver checkpoint and can resume after a Lack of Progress in any orientation, relocalising itself by matching sensed walls against the saved map

1) INTRODUCTION

Our team comprises four members — Abhijit Sharma, Aaryaman Talwar, Kashvi Khajanchi and Jasjyot Singh Sethi bringing together a blend of unique perspectives and experiences, united by a shared goal: to collaborate and deliver the best possible outcome. Our journey toward the RoboCup Junior Rescue Maze began in our robotics lab, and our participation in earlier RoboCup events gave us insight into the level of precision and reliability an international arena demands.

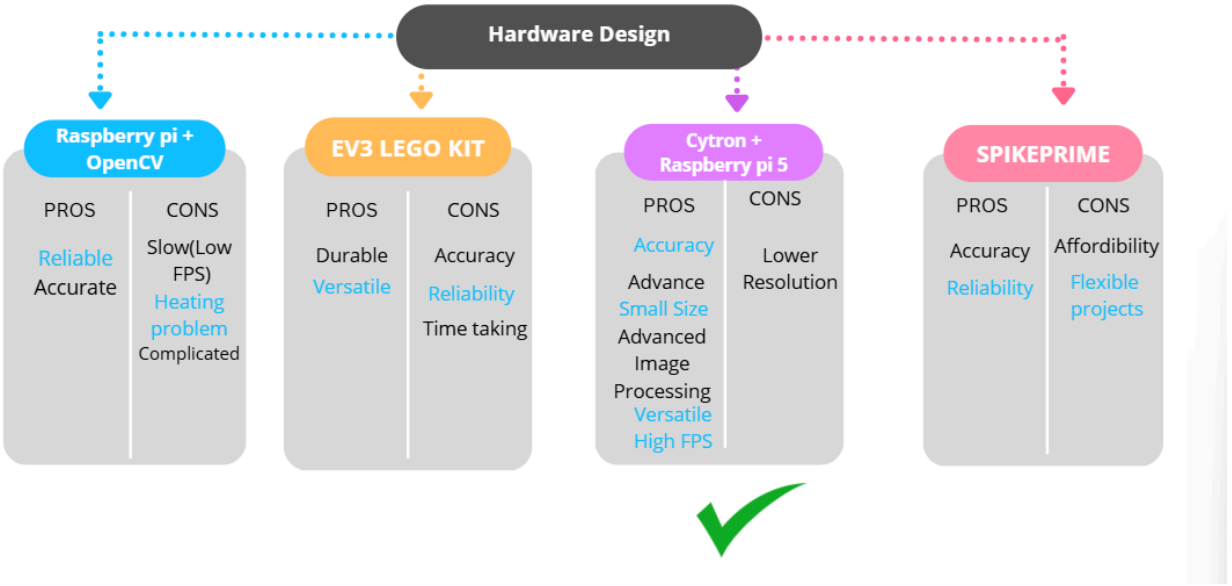
We divided tasks based on our strengths, yet our roles constantly overlapped as we worked as a unified team , whether designing the chassis, writing firmware, building the mapping engine, or testing in our practice maze. The competition has not only expanded our technical knowledge but also sharpened essential 21st-century skills like collaboration, problem-solving and critical thinking.

The mission: explore and map an unknown tiled maze (300 mm tiles, 150 mm walls) within an 8-minute run, identify victims, handle coloured hazard tiles, traverse ramps between floor levels, and return to the start tile for the exit bonus. Scoring rewards coverage, victim identification, checkpoint visits, blue-tile stops, ramp navigation and reliability, every Lack of Progress subtracts from the reliability bonus, so our entire design philosophy is built around never needing one.

Hazard tile	Colour	Rule behaviour
Hole	Black	Entering = automatic LoP. Must be avoided entirely.
Puddle	Blue	Mandatory 5 s stop on every visit (skipping = LoP). +30 first visit, -10 each revisit.
Checkpoint	Silver	Restart point after a LoP. +10 per visit.
Danger zone	Red	Entrance marker; higher-value zone beyond.

2) PROJECT PLANNING

Our objective throughout has been a cost-effective yet efficient robot. With each trial we analysed every aspect of the build and reworked it to best fit the requirements, evaluating candidate hardware on cost, accuracy, availability, reliability and our own competence to use it. The project ran in deliberate phases: platform bring-up, sensor selection (including alternatives we tested and rejected - see fig.2a), a complete control-loop overhaul, the mapping and routing engine, and finally multi-floor ramps and checkpoint recovery. Every subsystem was tested in isolation before integration, and a known-good legacy firmware/software stack is always preserved and restorable with one command.



(fig - 2a)

2a. Project Timeline

Timeline	Work done
Phase 1	Chassis build — • 4-motor differential drive • Cytron Motion 2350 Pro bring-up • motor mapping • inversion calibration
Phase 2	Sensor trials — • HC-SR04 ultrasonics and VL6180X tested and rejected • standardised on 6× VL53L0X behind a PCA9548A I²C mux
Phase 3	Raspberry Pi 5 integration — • UART protocol • telemetry parser • live HTTP dashboard
Phase 4	Control-loop overhaul — • blocking reads removed • continuous ranging • ~5 Hz → ~70 Hz
Phase 5	Motion primitives — • absolute-cardinal turns • calibrated MOVE_TILE with residual carry • PD wall-follow
Phase 6	Mapping engine — • frontier exploration • turn-aware Dijkstra • hazard-tile rules • checkpoint persistence
Phase 7	• Long-corridor blind odometry • ramp traversal • multi-floor mapping
Phase 8	• LoP resume with full relocalisation • chained moves • reliability hardening
Phase 9 (ongoing)	• OpenMV victim-detection camera • rescue-kit deployment • power-system upgrade (3S) • competition rehearsal

(Figure – 2b Project Timeline)

3) SYSTEM ARCHITECTURE

The core design philosophy and what keeps the robot debuggable is a strict separation of concerns across three tiers:

Tier	Hardware	Responsibility
Real-time body	Cytron Motion 2350 Pro (RP2350, dual-core Cortex-M33)	All sensor I/O, motor PWM, closed-loop motion primitives. Executes ONE command at a time and acknowledges it. Holds no map, makes no decisions.
Strategy mind	Raspberry Pi 5 (4 GB, Debian)	Telemetry parsing, trigonometric sensor correction, mapping + exploration + routing, live HTTP dashboard on port 80.
Development	Laptop	SSH code push and monitoring only — absent during a run.

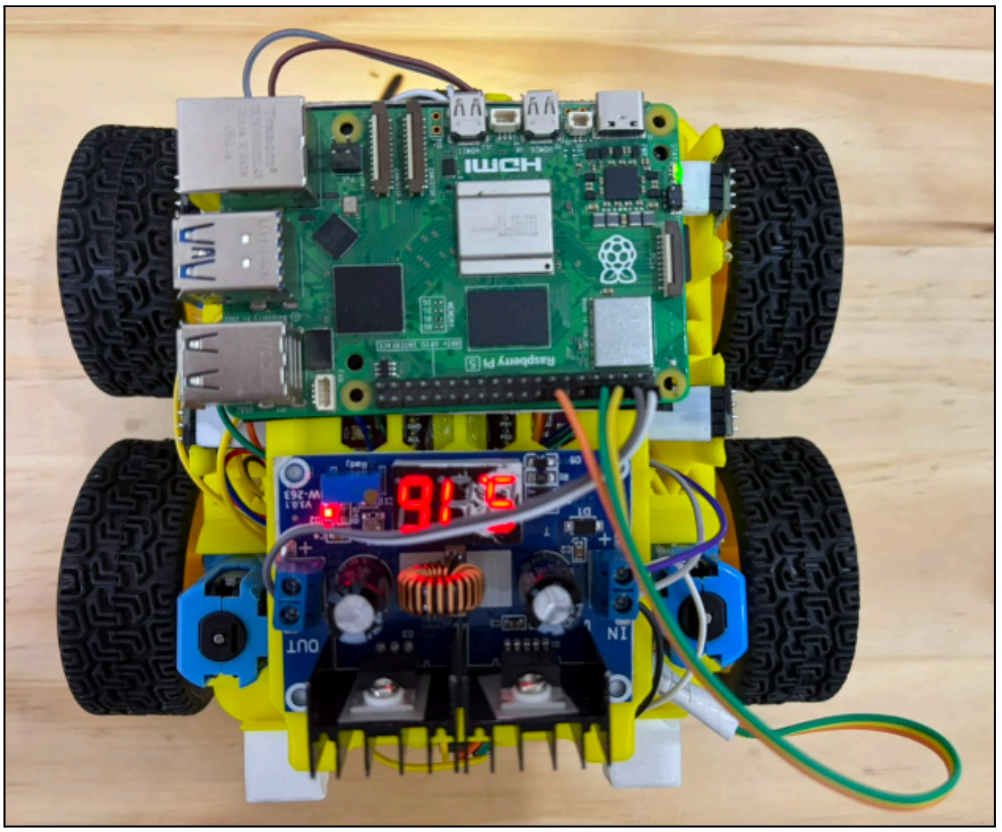
The boards are linked by wired UART (115200 8N1). The firmware streams one ASCII telemetry line every 50 ms (20 Hz); the Pi sends single-line ASCII commands, and every motion command terminates with **ACK <name> ok** or **ACK <name> fail reason=…**. The mapper enforces a strict one-command-in-flight, ACK-gated loop with name-aware ACK pairing, so a stray acknowledgement can never be mistaken for a motion result.

(fig - 3a)

4) HARDWARE

4a. Chassis and Drive

Property	Value
Body size	186 mm long × 170 mm wide (146 mm chassis + 20 mm bumper spacers front and back)
Turning diagonal	252 mm — fits the 280 mm minimum pathway with margin
Drive	4 brushed gear motors, differential (tank) drive; pivots in place
PWM limits	45 (gearbox stall floor) to 255
Controls	2 push buttons: tap = pause/resume (LoP procedure), 5 s hold = start/stop
Indicators	2× NeoPixel: pixel 0 = referee-facing regulation blink, pixel 1 = team status cue

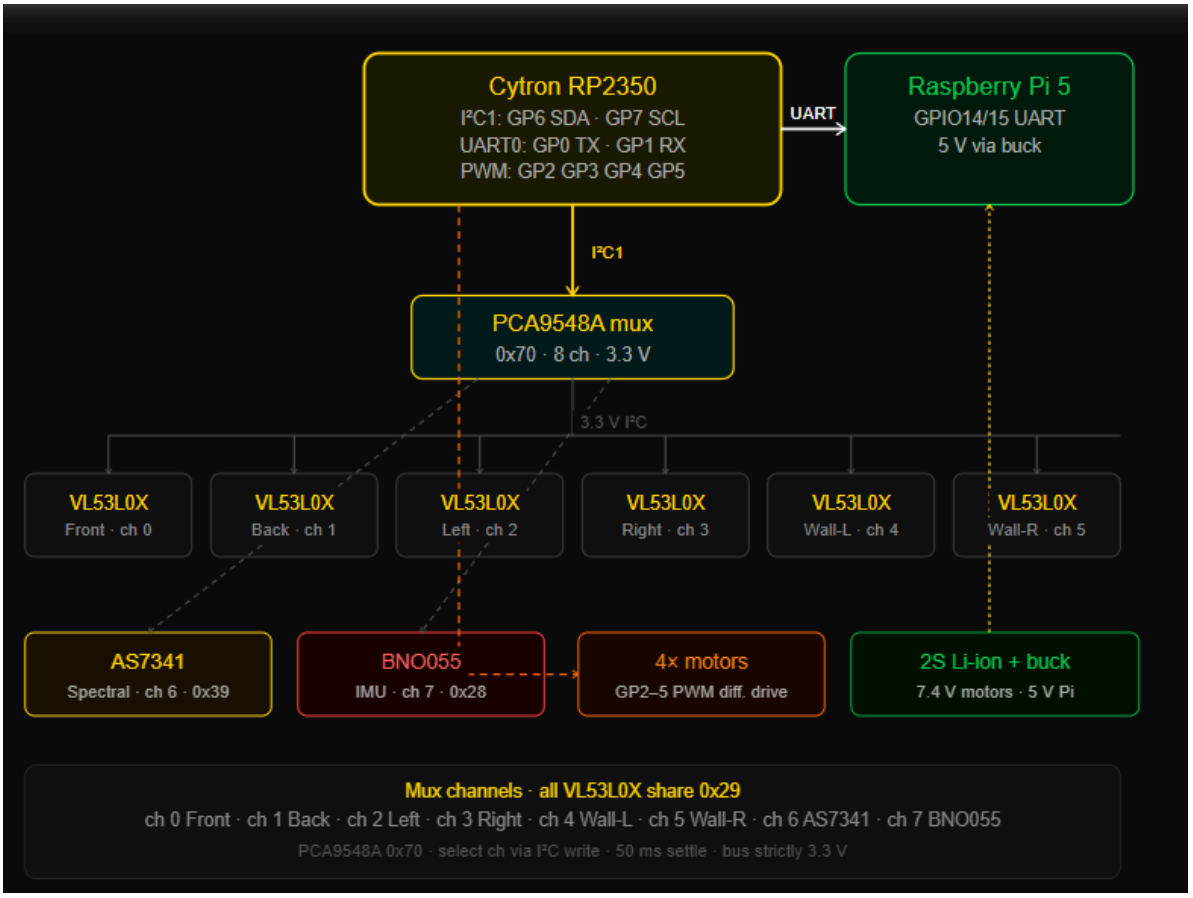


(fig – 4a)

4b. Electronics and Wiring

All sensors sit on the RP2350's I²C1 bus behind the multiplexer, run at 100 kHz with a 50 ms settle after channel select (VL53L0X initialisation is unreliable at 400 kHz through the mux). The bus is strictly 3.3 V, a lesson learned after a 5 V level-shifted breakout killed a sensor and a dead ToF once clamped SDA low and jammed the entire bus.

Qty	Component	Role
1	Cytron Motion 2350 Pro (RP2350)	Main microcontroller + motor driver
1	Raspberry Pi 5 (4 GB)	Strategy computer + dashboard
1	PCA9548A I²C multiplexer	Required — all six VL53L0X share address 0x29
6	VL53L0X time-of-flight	Front / back / left / right mapping + 2 dedicated wall-follow
1	AS7341 spectral sensor	Floor colour: black / blue / silver / red / normal
1	BNO055 IMU	Heading (gyro+accel fusion) and tilt for ramps
4	Brushed gear motors	Differential drive (2 left / 2 right)
1	2S Li-ion + XL4016 buck	7.4 V motors; regulated 5 V for the Pi
1	OpenMV camera (planned)	Side-mounted victim vision — integration in progress



(figure - 4b)

4c. Power System

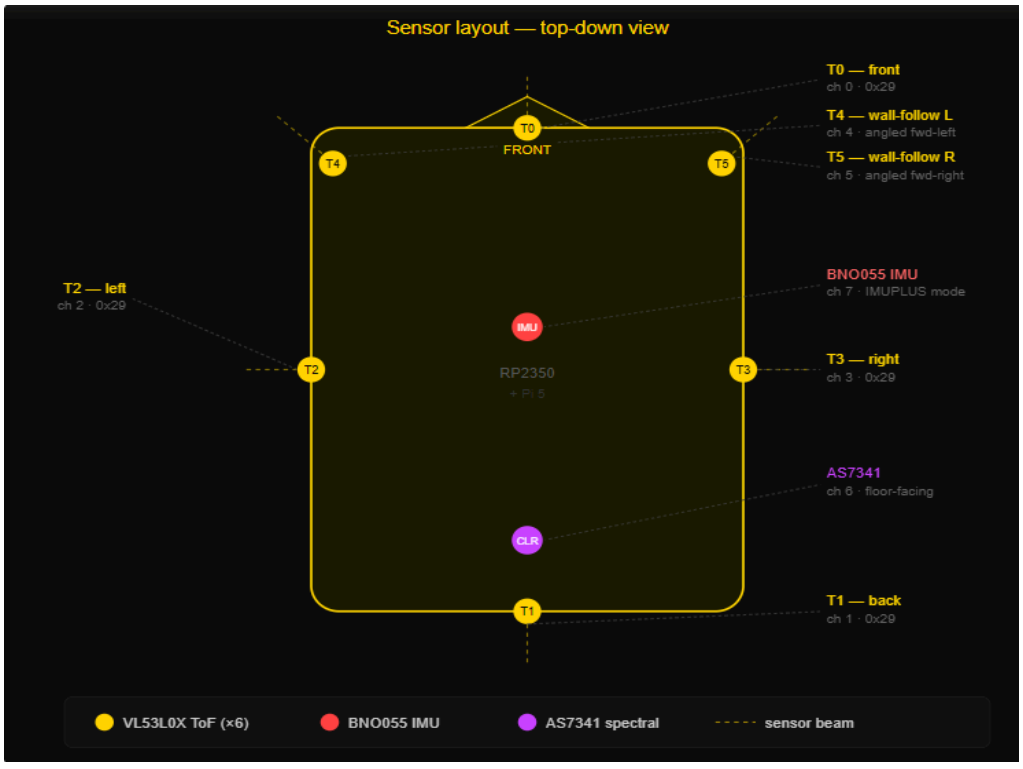
A 2S Li-ion pack (7.4 V, charged to 8.4 V before runs) feeds the motors directly and an XL4016 buck converter (trimmed to 5.2 V) for the Pi, with a common ground for the UART. Power proved to be our single biggest reliability constraint: motor stall current during wall rams sags the rail, and the Pi can

brown out under combined load. We mitigated this in software, lower ram PWM, fewer rams, sensor-verified moves, and a two-sample yaw reference that detects an IMU reset caused by a brown-out , and a 3S upgrade with bulk capacitance is planned. This constraint shaped the whole design: the software is deliberately conservative and verifies every motion with sensors rather than assuming success.

4d. Sensor Suite

We did not arrive at the final suite on the first attempt; the rejected alternatives shaped the design:

- **HC-SR04 ultrasonics (rejected):** originally fitted for wall-following, but the blocking echo timing starved the control loop, the RP2350 GPIO is not 5 V tolerant, and the wide beam blurred the wall edges needed for alignment.
- **VL6180X short-range ToF (rejected):** ideal range on paper for a wall ~94 mm away, but the modules repeatedly failed to initialise on the 3.3 V bus and one unit died.
- **VL53L0X standardisation (adopted):** one chip across all six distance positions — one driver, one set of quirks, interchangeable spares. Its long-range quirks (8191 mm 'no echo', ~30 mm dead zone, out-of-range spikes, 1/cos(tilt) slant inflation) are each engineered around in software.
- **BNO055 in IMUPLUS mode:** magnetometer OFF — mag calibration is impossible next to four brushed motors, and the maze only needs relative 90° heading. The gyro gives usable yaw even at calibration 0. • **AS7341 spectral sensor:** 11 channels give robust black/blue/silver/red separation via nearest-reference classification against EEPROM-stored calibration samples; integration time tuned from ~550 ms to ~110 ms per read.



(fig- 4c)

5) SOFTWARE

5a. Sensor Acquisition

Our original loop performed six blocking ToF reads plus a blocking colour read every iteration — a ~250 ms loop. Every behaviour sampled at ~5 Hz, causing tile overshoot, centering oscillation and erratic turns. The overhaul moved all six VL53L0X into hardware continuous-ranging mode (the loop just polls 'ready?'), made the colour read fully asynchronous, and rate-limited telemetry to 20 Hz. A sensor producing no data for 400 ms is flagged offline and automatically re-initialised — self-healing, and the loop can never block on a wedged chip. Result: a ~70 Hz control loop with millimetre-accurate stops.

5b. Motion Primitives & Heading Control

The firmware exposes closed-loop primitives — TURN, MOVE_TILE, RAM, CENTER, WAIT_BLUE — and the key heading insight is that turns target **absolute grid cardinals**, not relative 90° steps: an off-axis entry is corrected by the turn instead of accumulated (relative turns once desynced the map until the robot 'phased through a wall'). A grid yaw reference is captured at power-on and drift is managed in layers: an in-place re-square every 3 tiles, and every 10 tiles a full recalibration — ram a known wall to physically square the chassis, re-anchor the yaw reference, recenter.

MOVE_TILE drives exactly one calibrated tile using the front ToF shrinking or back ToF growing, with a residual-error carry so each move cancels the previous one's leftover; a stop at a front wall runs an absolute alignment that zeroes the error. Tile size itself is calibrated live from corridor runs (yaw-gated, band-limited 250–360 mm, running-averaged), so the robot adapts to any arena. A phantom-tile guard refuses to start a move when the front reads ≤ 150 mm — geometrically the bands of 'my boundary walled' vs 'next boundary walled' can never overlap it.

5c. Wall Following

Lateral steering uses the two dedicated side ToFs with a PD controller (bias = Kp·error + Kd·Δerror, clamped ±40 PWM, applied differentially) and **per-side targets** — the two sensors legitimately read different distances at the true centre, and a shared target steered into a wall. Wall presence is debounced asymmetrically (500 ms to assert, 100 ms to release) and a wall must persist 750 ms before its loss triggers any reaction, so a single flaky frame can neither invent nor delete a wall.

5d. Blind Odometry

In a 7-tile corridor both end walls are beyond reliable ToF range, so a move can begin with no valid travel reference. Blind mode drives at fixed PWM with a field-calibrated time-based travel estimate, re-anchors whenever an established side wall ends (wall edges sit exactly on tile boundaries), and snaps the stop target onto the grid the moment a front wall comes into range — an absolute re-anchor that absorbs the accumulated time error.

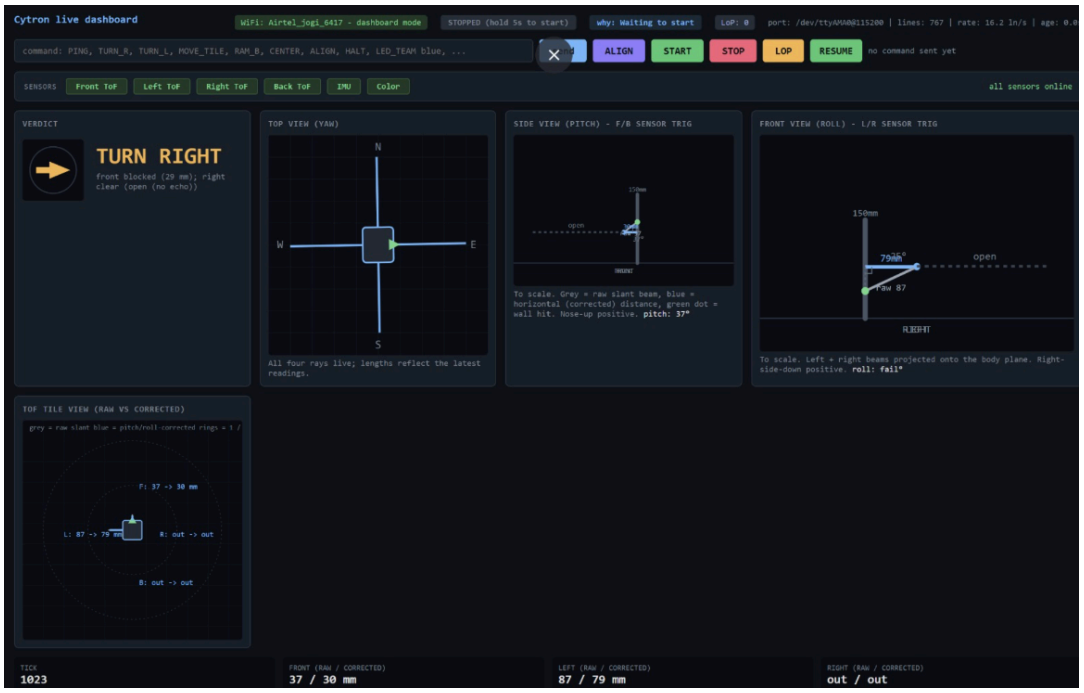
5e. Ramps

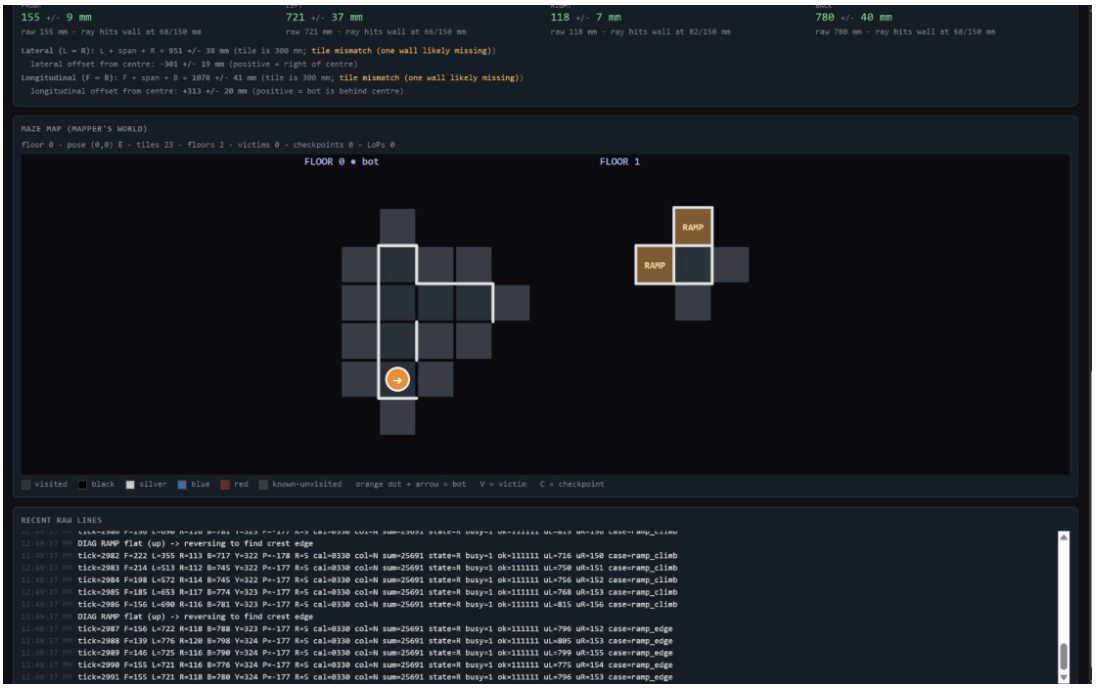
A ramp is distinguished from an obstacle by a sustained pitch change (>15°) with minimal roll change (<6°). Going up, the robot climbs on tilt-corrected wall-follow until the floor is flat, then reverses until pitch returns — at that instant the rear wheel contact sits exactly on the crest edge, a known tile boundary. From the edge the chassis geometry gives an exact advance to the next tile centre, and the traversal acts as an absolute position re-anchor. RAMP UP/DOWN events drive the

mapper's floor transitions.

5f. Mapping & Exploration

The Pi holds a WorldMap of (floor, x, y) → tile {colour, walls, victims, checkpoint}, with walls recorded reciprocally on both adjacent tiles and ramps stored as directed links between floors. Exploration runs a SENSE → DECIDE → ACT loop: distances are median-filtered over an adaptive window (a momentary sensor flash cannot corrupt the map), the planner takes any open edge to an unexplored neighbour first (unknown-first, least-turning), otherwise routes to the nearest frontier, and when no frontiers remain it routes home for the exit bonus.





(figures 5a and 5b)

5g. Routing — turn-aware Dijkstra

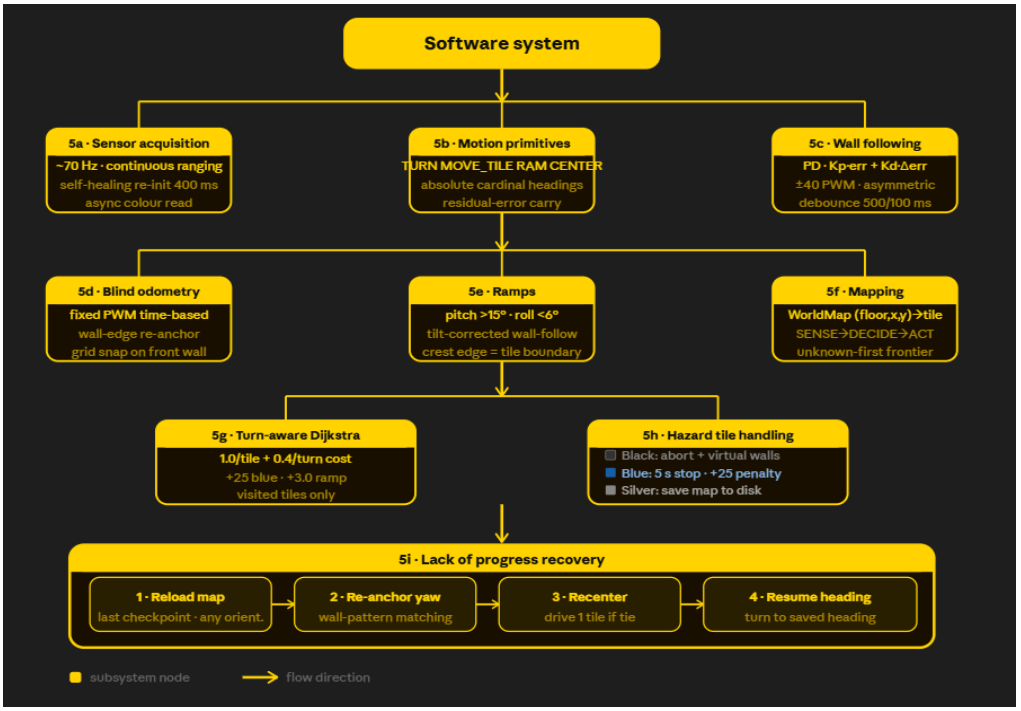
Paths are planned over (cell, heading) states with cost 1.0 per tile plus 0.4 per 90° pivot — among equal-length routes the planner picks the one with the fewest turns, which is both faster and more reliable since every pivot is a fresh chance to pick up yaw error. Rule-aware costs are encoded directly: +25 to re-enter a visited blue tile (a revisit loses 10 points and forces another 5 s stop) and +3.0 per ramp link. Routing only crosses visited, non-black tiles, so the robot returns home across floors through known territory.

5h. Hazard & Tile-Colour Handling

- **Black:** the firmware aborts mid-move and reverses to the previous tile centre; the mapper boxes the tile with virtual walls on all four sides so it is never attempted again from any direction.
- **Blue:** LED indication + mandatory 5 s stop; the pause flag re-arms on leaving so any re-entry serves a fresh 5 s; the routing penalty makes revisits a last resort.
- **Silver:** checkpoint — white LED flash, and the entire map is atomically written to disk so a restart can resume. • **Red:** recorded and logged (the rules attach no behavioural rule).

5i. Lack of Progress

After a LoP the captain may place the robot on the last checkpoint in any orientation. On resume the mapper reloads the persisted map, re-anchors the yaw frame, recenters, and derives which cardinal it faces by matching a median-sensed wall pattern against the checkpoint tile's known walls — genuine ties are resolved by driving one tile and re-matching. It then physically turns back to the heading held when the checkpoint was saved, so the internal model and the body always agree before exploration continues.



6) INNOVATIONS

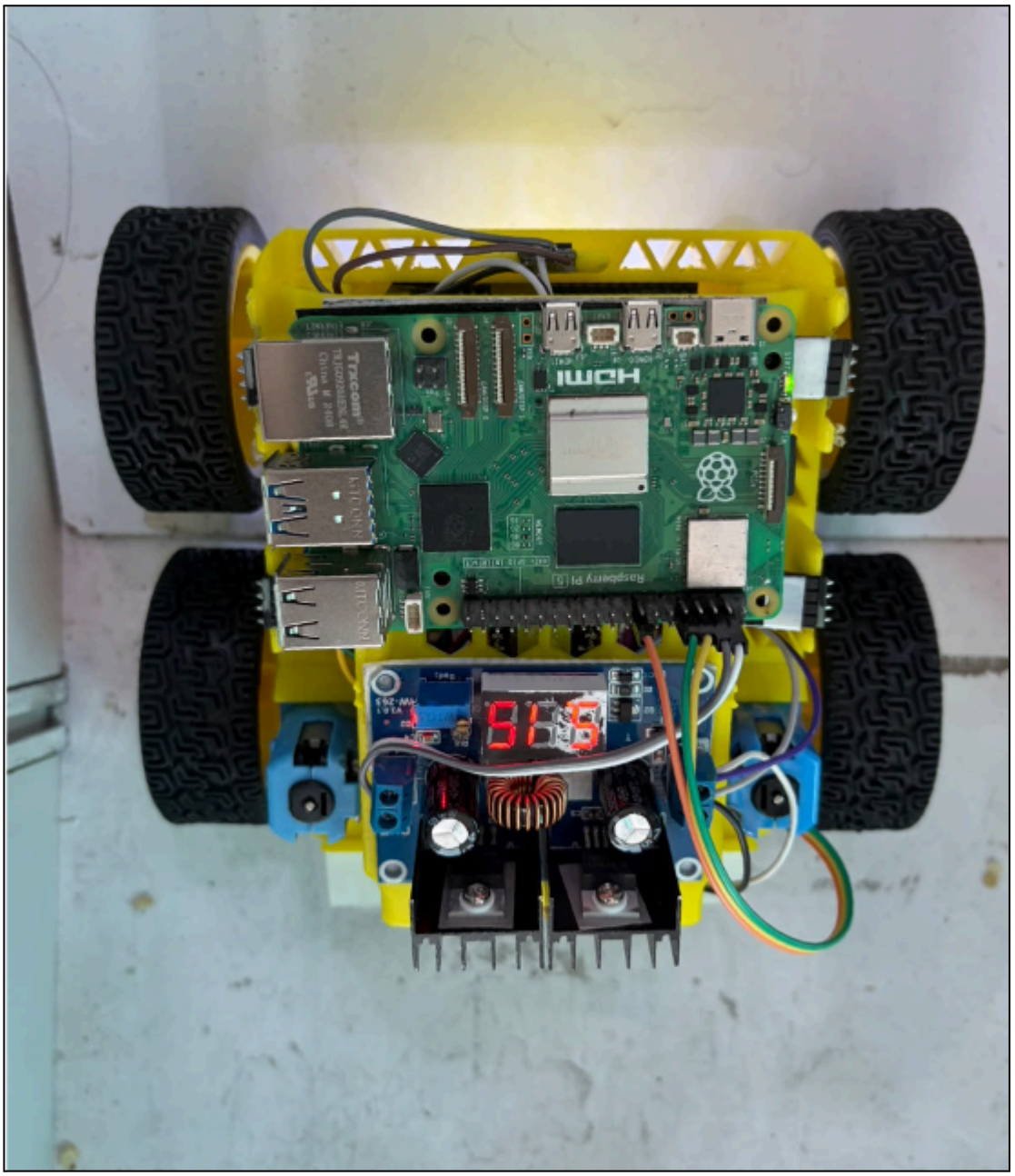
- **ACK-gated body/mind split:** one command in flight, every motion verified, name-aware acknowledgement pairing — the architecture itself is our main reliability feature.
- **Self-healing sensor pipeline:** continuous ranging + automatic re-initialisation of any sensor that goes silent, without ever blocking the 70 Hz loop.
- **Live tile-size calibration:** the robot measures the arena's actual tile pitch from corridor runs, yaw-gated and running-averaged — no per-arena reconfiguration.
- **Geometry-gated sensor correction:** every distance reading is tilt-projected and rejected if the beam ray would clear the wall top or hit the floor — a shaken or ramp-climbing robot never draws phantom walls.
- **Blind odometry with edge re-anchoring:** time-based travel fused with wall-edge events and grid snapping survives corridors longer than the sensor range.
- **Scoring-aware routing:** the rulebook's point values are encoded as edge costs, so the planner natively avoids blue-tile revisits and unnecessary floor changes.
- **Orientation-free LoP recovery:** scored wall-pattern relocalisation means the captain can place the robot any way round and it sorts itself out.
- **Chained moves:** known corridors are driven without full stops, naturally slowing near each tile centre — exactly when the future victim cameras get their best look at the walls.

7) PERFORMANCE EVALUATION AND CONCLUSION

Every subsystem was validated both off-bot and on-bot. The complete mapping algorithm runs against a simulated maze with fake firmware callbacks, asserting full coverage, correct hazard handling and return-home; a port of the same algorithm passes 40/40 randomly generated mazes headless. On the robot, a bring-up checklist verifies telemetry and sensor health, single-turn accuracy (≤ 2° error), corridor centering, the colour-tile drills (blue slow-and-stop, silver flash, black box-out), the full LoP drill, and ramp climbs onto the dashboard's second floor.

Rich diagnostic telemetry (turn completion error, move position error, blind-mode transitions, ramp phases) plus the live dashboard let us post-evaluate every run and measure each fix. The headline result of this process is the control-loop overhaul — from ~5 Hz to ~70 Hz — which transformed the robot's behaviour from 30–50 mm tile overshoots and oscillating centering to millimetre-accurate stops. The problems table in our engineering log (brown-outs, wall-follow controller iterations, heading desync, phantom tiles, jammed I²C bus) reads as the true history of the design: nearly every feature described in this paper exists because a specific failure demanded it.

We look forward to showcasing our robot in the actual arena and to exchanging ideas with competing teams about the engineering behind their robots.



(fig-7a)

8) ACKNOWLEDGEMENTS

We extend our sincere gratitude to the RoboCup organisation for the opportunity to participate and interact with the best of minds. A special thanks to our mentor, Mr. Vivek Gautum, for his unwavering guidance and support at every step, and to The Makers Robotics for providing the lab, equipment and environment that made this build possible. We are grateful to our teachers for helping us balance academic commitments with the competition, and finally to our parents for making such experiences and learning opportunities part of our lives.

9) APPENDIX

- **References — Hardware**

Cytron Motion 2350 Pro (RP2350), Raspberry Pi 5, VL53L0X , AS7341, BNO055 , PCA9548A , WS2812 NeoPixel , XL4016

- **References — Software**

Arduino (Earle Philhower RP2040/RP2350 core) , Python 3 , Adafruit sensor librarieS, Github

Parameter	Value	Meaning
Tile nominal / accept band	300 / 250–360 mm	calibrated tile pitch
Wall detect threshold	180 mm	corrected distance below = wall
ToF odometry trust limit	800 mm	beyond = never a travel reference
Control loop	~70 Hz	continuous-ranging pipeline
Turn tolerance / confirm	2° / 150 ms	turn-done criterion
Motor PWM floor / max	45 / 255	gearbox stall / full
Wall debounce assert / release	500 / 100 ms	anti-glitch / quick drop
Routing turn cost	0.4 tile / turn	turn-aware Dijkstra
Blue revisit penalty	+25 tiles	scoring-aware routing
Ramp detect	>15° pitch, <6° roll	ramp vs obstacle
Re-square / full recal	3 / 10 tiles	drift management
Blue pause	5 s	rule-mandated stop